AqStoqFlow Payroll Trust Spine Closure

Date: 2026-08-21 Run type: source-aligned architecture review and completion roadmap Implementation decision: `DESIGN_READY_IMPLEMENTATION_NOT_STARTED` 012 decision: `NOT_APPROVED_FOR_DOWNSTREAM_RATIFICATION` Production decision: `NO_GO`

Executive decision

The Payroll Trust Spine is the correct next high-impact internal platform program. The live source confirms that payroll run review, approval, payslip emission, and accounting posting are still collapsed into one command and one principal final event. The current 14/14 payroll presence gate does not detect that bypass.

This run did not edit the payroll boundary. The exact schema, service, action, gate, test, and UI files required by the prompt already contain large uncommitted user-owned changes. The prompt explicitly requires preservation of those changes and says to stop when overlapping ownership cannot be proven safe. The safe result of this run is therefore a source-backed architecture, correction to the prompt, and a phase-gated completion roadmap.

The implementation should begin only after Work Package 0 establishes a controlled branch/worktree or an explicit ownership snapshot for the overlapping files.

Current repository truth

Evidence	Current result	Meaning
Dirty worktree	Broad and overlapping	`prisma/schema.prisma`, payroll control service/schemas/actions/tests, payroll gate/tests, and payroll UI files contain user-owned changes.
Prisma validation	Pass	The current dirty schema parses; this is not migration deployment proof.
TypeScript typecheck	Pass	The current dirty tree typechecks.
Payroll presence live replay	Ready, 14/14	Static token-presence result only; it does not prove the required lifecycle.
Report-trust live replay	Blocked, 34/35	`signed_customer_statement_external_access_foundation` is a gate false negative: the token implementation was safely delegated to `signed-external-access-token.ts`, while the gate still requires inline HMAC implementation markers.
Offline POS live replay	Ready, 16/16	No regression found.
Focused seven-file Jest baseline	Not verified	The command produced no test result within the bounded run and was interrupted.
Isolated payroll gate Jest file	Not verified	The single JavaScript gate test also produced no result within the bounded run and was interrupted.

Commands run:

```
npm run prisma:validate                          PASS
npm run typecheck                                PASS
direct payroll gate replay                       READY 14/14
direct report-trust gate replay                  BLOCKED 34/35
direct offline-POS gate replay                   READY 16/16
focused seven-file Jest baseline                 NO RESULT; INTERRUPTED
isolated payroll-presence gate Jest test         NO RESULT; INTERRUPTED
```

No saved 35/35 report-trust artifact may override the newer live replay. The blocker is not evidence of a broken customer-statement token implementation: source inspection confirms the shared helper still enforces a minimum 32-character secret, HMAC-SHA256 signing, and timing-safe verification. It is evidence that the gate is coupled to a prior implementation shape.

Confirmed payroll defects

1. `approveAndPostPayrollRun` accepts both `CALCULATED` and `REVIEWED`.
2. The same transaction creates emitted payslips, posts the ledger, and moves the run directly to `POSTED`.
3. One actor is persisted as approver, emitter, and poster.
4. The run version is not used in a compare-and-set transition claim.
5. The transaction uses the ordinary transaction helper rather than the existing serializable helper.
6. Approval has no distinct `PAYROLL_RUN_APPROVED` business event.
7. Payslip emission is an outbox notification nested under `payroll.run.posted`, not an independently durable semantic event.
8. Leave approval updates the request and leave-balance ledger but records no canonical `LEAVE_APPROVED` event.
9. Declaration preparation lacks `freshAuth: true` at the protected-action boundary.
10. The gate can remain 14/14 with all the above defects present.
11. Sensitive-action policies exist for payroll approval and payment stages, but not for run review, payslip emission, or run posting.
12. The legacy alias for `payroll.runs.review` currently includes read-oriented permissions (`PAYROLL_READ`, `PAYROLL_REPORTS_READ`). That mapping is not safe for a new state-changing review command without an explicit RBAC decision.

What the attached prompt needs to do better

1. Replace the big-bang run with phase gates

The prompt combines schema, backfill, service refactoring, authorization, events, close assurance, data trust, UI, migration deployment, concurrency certification, and cross-domain ratification in one execution. Those are dependent release units, not one safe edit.

Use the work packages below. Each package must produce its own evidence and must not begin until the prior decision gate passes.

2. Resolve the legacy-backfill contradiction

The proposed transition row requires non-null `fromStatus`, `actorId`, `businessEventId`, and evidence hashes, while the backfill rules correctly forbid fabricated actors, prior states, events, or fresh-auth proof. Both requirements cannot be satisfied for historical combined transitions.

The schema must distinguish runtime proof from legacy snapshots:

- `origin`: `RUNTIME` or `LEGACY_BACKFILL`;
- `evidenceStatus`: `VERIFIED` or `LEGACY_PARTIAL_EVIDENCE`;
- nullable `fromStatus`, `actorId`, and `businessEventId` only for `LEGACY_BACKFILL` rows;
- a database check requiring all authoritative fields for every `RUNTIME` row;
- a legacy snapshot row that records only facts actually provable from stored history.

Do not backfill a fake sequence of review, approval, emission, and posting transitions from one old combined timestamp.

3. Enforce append-only at the database boundary

A table is not append-only merely because the service avoids updates. Add a PostgreSQL trigger that rejects transition updates and deletes. Add check constraints for allowed runtime transitions and runtime evidence completeness.

4. Stop assuming a tenant payroll SoD policy already exists

No tenant-configurable payroll-run SoD policy was found. The current sensitive-action layer offers a generic self-approval check, not a versioned payroll control policy.

For the first safe release, use a documented strong baseline and no silent exception:

- preparer must differ from reviewer;
- approver must differ from preparer and reviewer;
- poster must differ from preparer and approver;
- reviewer may also post, which permits a three-person human control model;
- emitter may be an authorized system worker or an authorized human; emission authority is distinct even when the human is not required to be a fourth person;
- payment requester, approver, and releaser remain three distinct actors;
- correction runs use the same separations.

If tenant-specific exceptions are commercially necessary, add a separately approved, versioned, effective-dated policy with maker-checker evidence. Do not hide an exception in metadata or role naming.

5. Repair RBAC before enabling the review mutation

`payroll.runs.review` must be classified as high or critical risk and must not inherit from read-only legacy aliases. Assign it to an explicit payroll checker/reviewer role and add negative RBAC tests before the command becomes callable.

6. Add a typed event-contract registry

The generic event schema currently accepts any event string and unknown payload. A typed event contract cannot be proven by string presence alone.

Add a payroll/HRIS event-contract module containing Zod payload schemas, canonical names, versions, idempotency rules, safe metadata, and compatibility transport topics. Reuse `BusinessEvent` and `BusinessEventOutbox`; do not create another universal event table.

7. Avoid a sensitive, oversized payslip manifest event

Requiring every payslip ID, number, archive reference, and employee identifier in a generic batch event creates privacy and payload-size risk.

Preferred v1 contract:

- one tenant-scoped `PAYSLIP_EMITTED` batch event per run transition;
- payload contains run ID, count, totals, and a deterministic manifest hash, not person identifiers;
- each payslip stores the shared batch event ID plus its existing document hash;
- the manifest is recomputed from tenant-scoped immutable payslip rows;

- one outbox message announces batch completion;
- no salary, bank, identity-document, or employee contact data enters the outbox.

If downstream delivery truly requires per-payslip events, introduce them as a separately load-tested contract rather than multiplying events inside the initial control repair.

8. Specify payroll-period semantics

The run lifecycle and payroll-period lifecycle are not identical. The implementation must not regress an already posted period when a correction run is approved.

- ordinary primary-run approval may move the period to `APPROVED`;
- ordinary primary-run posting may move it to `POSTED`;
- review and emission do not change the period status;
- correction, bonus, and complementary runs must not move a posted or paid period backward;
- payment release remains `POSTED -> PAID`, never `PAID -> POSTED`.

9. Make cutover and rollback fail closed

Removing the combined action is not enough. Rolling back to old application code after cutover would restore the bypass.

Use two additive database deployments:

1. schema/ledger deployment, compatible with the current application;
2. cutover enforcement after the staged commands are verified, including a database transition guard.

The rollback mechanism is a server-owned write-disable switch and forward fix, not reactivation of `approveAndPostPayrollRun`. Old application code must fail closed against the cutover transition guard.

10. Separate static contract proof from transactional proof

Static source markers cannot prove atomicity, rollback, or concurrency. The live report-trust false negative demonstrates why implementation-string gates are brittle.

The final payroll readiness decision must aggregate three independent layers:

- contract/static gate: command names, action permissions, forbidden combined exports, typed event registry, migration presence;
- focused behavior suite: invalid transitions, SoD, fresh auth, tenant scope, idempotency, failure injection;
- disposable PostgreSQL certificate: real concurrent transactions, serializable retry behavior, database constraints/triggers, rollback, migration/backfill rerun.

Target lifecycle contract

From	Command	To	Permission	Fresh auth	Mandatory SoD	Canonical event	Close/data-trust effect
`CALCULATED`	`reviewPayrollRun`	`REVIEWED`	`payroll.runs.review`	Yes	reviewer != preparer	`PAYROLL_RUN_REVIEWED v1`	Invalidate certified evidence if this run changes a certified period; otherwise expose pending reviewed payroll.

From	Command	To	Permission	Fresh auth	Mandatory SoD	Canonical event	Close/data-trust effect
REVIEWED	approvePayrollRun	APPROVED	payroll.runs.approve	Yes	approver != preparer/reviewer	PAYROLL_RUN_APPROVED v1	Invalidate affected certified close; expose approved-unemitted liability.
APPROVED	emitPayrollPayslips	EMITTED	payroll.payslips.emit	Yes for human; attested worker policy for system	governed emitter; no approval bypass	PAYSLIP_EMITTED v1 batch	Invalidate affected certified close; expose emitted-unposted run as blocker.
EMITTED	postPayrollRun	POSTED	payroll.runs.post	Yes	poster != preparer/approver	PAYROLL_POSTED v1	Preserve balanced journal/source link and PAYROLL_RUN_POSTED invalidation.

All other direct transitions fail with `INVALID_TRANSITION`. A correction run follows the same four transitions and never mutates the original run or payslip.

Atomic command algorithm

Every runtime transition uses the same transaction kernel:

1. Parse a stage-specific Zod input.
2. Accept only resource ID, idempotency key, optional correlation ID, and stage-specific evidence from the client.
3. Derive tenant, actor, permission, module access, and fresh-auth evidence from the protected server context.
4. Start a serializable database transaction with bounded retry for serialization conflicts.
5. Check the transition ledger by tenant and idempotency key.
6. Return a replay only for the same payload hash; otherwise fail with `IDEMPOTENCY_CONFLICT`.
7. Load the run by organization and run ID.
8. Verify exact source status, expected version, stage prerequisites, and SoD.
9. Create stage artifacts and durable event/outbox evidence inside the transaction.
10. Compare-and-set the run by organization, ID, source status, and version; increment version once.
11. Write the transition ledger row, audit, and any close invalidation in the same transaction.
12. Mark the event applied only after the aggregate, transition, audit, and invalidation writes succeed.
13. If compare-and-set affects zero rows, roll back all provisional writes and return `CONCURRENCY_CONFLICT`.

For two concurrent commands using the same key and payload, one transaction creates the transition and the other returns an idempotent replay after bounded serialization retry. For different keys targeting the same state, exactly one commits and the loser receives `CONCURRENCY_CONFLICT`.

Corrected additive schema design

The exact Prisma syntax must follow current tenant-safe relation conventions, but the persistence contract is:

```
PayrollRunTransition
  id
  organizationId
  payrollRunId
  sequence
  fromStatus?                  nullable only for legacy snapshot
  toStatus
  actorId?                     nullable only for legacy snapshot
  occurredAt
  correlationId?
  idempotencyKey
  payloadHash
  evidenceHash
  businessEventId?             nullable only for legacy snapshot
  origin                       RUNTIME | LEGACY_BACKFILL
  evidenceStatus               VERIFIED | LEGACY_PARTIAL_EVIDENCE
  metadata?
  createdAt
```

Required database controls:

- compound tenant/run foreign key;
- compound tenant/business-event foreign key when an event is present;
- unique `(organizationId, payrollRunId, sequence)`;
- unique `(organizationId, payrollRunId, toStatus)`;
- unique `(organizationId, idempotencyKey)`;
- unique `(organizationId, businessEventId)` for non-null stage events;
- runtime evidence completeness check;
- allowed runtime transition-pair check;
- immutable update/delete trigger;
- supporting tenant/run/time and tenant/status/time indexes.

Add to `PayrollRun` only fields used by new runtime transitions:

- `reviewedAt`;
- `reviewedBusinessEventId`;
- `approvedBusinessEventId`;
- `emittedBusinessEventId`;
- transition relation.

For deterministic batch emission, `PayrollPayslip.emittedBusinessEventId` is a non-unique tenant-scoped index because all payslips in a run share the batch event. Do not declare it unique.

Add `HrisTimeRequest.decisionBusinessEventId` for new decisions. Historical rows remain null unless an authoritative matching event already exists.

Do not update finalized historical payroll rows merely to populate nullable evidence links. Existing payroll immutability triggers must remain effective.

Canonical event contracts

Event	Aggregate	Idempotency key	Required evidence	Outbox
`LEAVE_APPROVED v1`	HRIS time request	`leave-approved:{requestId}`	request source hash, approval evidence hash, decision hash, balance-entry ID	safe employee/manager notification without raw reason or balance detail
`PAYROLL_RUN_REVIEWED v1`	payroll run	`payroll-run:{runId}:review:{targetVersion}`	calculation, attendance, country-pack, register, reviewer, SoD evidence hashes	payroll approval-ready notification
`PAYROLL_RUN_APPROVED v1`	payroll run	`payroll-run:{runId}:approve:{targetVersion}`	reviewed transition/event, approved calculation/evidence hash, approver and SoD proof	emission-ready notification
`PAYSLIP_EMITTED v1`	payroll run emission batch	`payroll-run:{runId}:emit:{targetVersion}`	payslip count, totals, deterministic manifest hash, country-pack and calculation hashes	one payroll batch-complete message
`PAYROLL_POSTED v1`	payroll run	`payroll-run:{runId}:post:{targetVersion}`	emission event/hash, ledger batch, journal, source link, document and component mapping hashes	accounting/close posting message

Compatibility dotted topics may remain as outbox event names. Canonical event types and their Zod schemas are authoritative.

Migration, backfill, cutover, and rollback

Schema deployment

- Add enums, transition table, nullable links, indexes, composite keys, and immutability controls.
- Do not change or delete existing payroll history.
- Run Prisma validation and migration safety checks.
- Replay on disposable PostgreSQL before any identified shared database.

Historical inventory and backfill

- Produce a read-only inventory by run status and evidence availability.
- For runs with authoritative stage-specific events, backfill only the transitions actually proven.
- For historical combined posted runs, create one `LEGACY_BACKFILL` snapshot row with `LEGACY_PARTIAL_EVIDENCE`; leave unknown prior state, actor, and event links null.
- Record counts, hashes, skipped rows, and reason codes.
- Make the operation idempotent and rerunnable.
- Treat new post-cutover missing evidence as a hard blocker; disclose legacy partial evidence without pretending it is newly verified.

Application cutover

- Add the four protected actions and update all callers.
- Remove the externally callable combined action and hook.
- Add review/emit/post sensitive-action policies.

- Repair the review permission aliases and risk classification.
- Add a fail-closed write-disable switch for incident rollback.
- Activate the database transition guard only after the new app path passes disposable PostgreSQL certification.

Rollback

- Never delete transition evidence.
- Never rewrite finalized payroll or payslips.
- Disable payroll lifecycle writes if a release must be rolled back.
- Do not restore the old combined action.
- Use a forward fix for service defects; retain additive schema.

Comprehensive execution roadmap

Work Package 0 — Baseline control and ownership

Classification: `blocked` until ownership is explicit.

Deliverables:

- commit or otherwise isolate the active user-owned payroll/payment work;
- create a controlled implementation branch/worktree from the intended baseline;
- record exact starting commit and dirty-tree disposition;
- repair the report-trust gate so it recognizes the shared signed-token helper or proves the helper behavior without inline-marker coupling;
- diagnose the Jest no-result condition and obtain a bounded single-test pass/fail result;
- regenerate live baseline reports without overwriting unrelated evidence.

Exit gate:

- exact payroll file ownership is known;
- report trust is honestly 35/35 or has a real product blocker;
- offline POS remains 16/16;
- one focused Jest test reliably exits.

Work Package 1 — Architecture and policy freeze

Classification: `surgical design repair`.

Deliverables:

- lifecycle ADR;
- SoD matrix and three-person baseline;
- typed event-contract schemas;
- payroll-period/correction semantics;
- cutover, rollback, and incident write-disable plan;
- transition contract manifest used by runtime and gates.

Exit gate: security, payroll, accounting, data, and migration reviewers approve one unambiguous contract.

Work Package 2 — Additive persistence and migration proof

Classification: `bridge/adapter`.

Deliverables:

- corrected transition ledger schema;
- nullable evidence links;
- compound tenant-safe foreign keys;
- DB checks and append-only trigger;
- migration test and disposable PostgreSQL replay;
- dry-run legacy evidence inventory.

Exit gate: zero-loss migration, rollback, tenant-negative, and rerun evidence pass.

Work Package 3 — Runtime transition kernel

Classification: `targeted refactor`.

Deliverables:

- shared serializable transition kernel;
- `reviewPayrollRun`;
- `approvePayrollRun`;
- `emitPayrollPayslips`;
- `postPayrollRun`;
- stable typed conflict/idempotency/SoD errors;
- combined command removed from the external action surface.

Exit gate: valid/invalid transition, idempotency, tenant, SoD, and failure-injection tests pass.

Work Package 4 — HRIS and event completion

Classification: `surgical repair`.

Deliverables:

- atomic `LEAVE_APPROVED` event/outbox/audit/balance decision;
- payroll review, approval, emission, and posting typed events;
- declaration preparation fresh auth with no current-time fallback;
- event-applied ordering after all material writes.

Exit gate: event removal, outbox failure, audit failure, and decision failure each roll back the transaction.

Work Package 5 — Close assurance and data trust

Classification: `bridge/adapter`.

Deliverables:

- approval/emission invalidation source codes only where certified-period evidence is affected;
- no duplicate invalidation on replay;
- emitted-unposted and missing post-cutover transition proof blockers;
- disclosed legacy partial-evidence state;
- updates to payroll analytics, assurance registry, accountant trust, and close projections that consume payroll evidence.

Exit gate: certified-close staleness and data-trust tests prove both hard blockers and legacy disclosures.

Work Package 6 — Read models and minimal operator UX

Classification: `surgical repair`.

Deliverables:

- exact next permitted action from the server;
- review/approve/emit/post capabilities and blockers;
- lifecycle evidence drawer;
- EN/FR strings;
- loading, empty, denied, step-up, conflict, legacy, degraded, success, and retry states;
- no Prisma/service imports in client code and no person-level leakage.

Exit gate: component/action tests plus authenticated EN/FR role-negative browser smoke pass.

Work Package 7 — Gate ratchet and PostgreSQL certification

Classification: `rebuild` of the shallow payroll gate, not the payroll domain.

Deliverables:

- contract-aware static checks;
- mutation fixtures for every prohibited bypass;
- focused behavior suite;
- real PostgreSQL concurrent reviewer/approver/emitter/poster tests;
- serializable retry, one-winner, replay, and payload-conflict proof;
- rollback injection after every material posting write;
- immutable transition and state-guard DB tests;
- signed machine-readable readiness artifact.

Exit gate: deleting or bypassing any invariant makes the readiness decision fail.

Work Package 8 — Cross-domain ratification

Classification: `validation/no-op unless regression`.

Required replay:

- Prisma validation and migration safety;

- typecheck and focused tests;
- payroll presence/trust gate;
- ledger close truth;
- report trust/export;
- policy-gate integration;
- offline POS replay;
- 012 architecture/completion decision;
- 013 data-trust/accountant portal decision;
- 014 offline POS decision only to confirm preservation.

Exit gate: 012 is approved on live evidence; 013 and 014 are not blocked by stale sequence reports.

Work Package 9 — Platform production completion handoff

Classification: `externally blocked release program`.

After internal payroll closure, production readiness still requires:

1. signed qualified country-pack review and independent verification;
2. certified payment rail and authority declaration sandbox evidence;
3. production-shaped tenant migration/backfill rehearsal and owner acceptance;
4. authenticated browser, accessibility, privacy, redaction, and tenant-negative evidence;
5. managed production database, secrets, credentials, monitoring, DR, incident, and support evidence;
6. clean candidate freeze and product/security/control owner approvals;
7. controlled pilot, Phase 2B decision, then Phase 3 decision.

Payroll Trust Spine closure is an internal technical prerequisite. It does not by itself certify statutory rules, production data, live disbursement, authority filing, accessibility, privacy compliance, or production release.

Before/after gate matrix

Invariant	Current 14/14 gate	Target gate
No `CALCULATED/REVIEWED -> POSTED` path	Not detected	Required static and behavior failure fixture
Separate transition commands	Not detected	Required
CAS and version increment	Not detected	Required plus PostgreSQL one-winner proof
Durable transition ledger	Not detected	Required plus DB immutability test
Typed canonical events	Not detected	Required contract registry and mutation tests
Declaration fresh auth	Not specifically detected	Required protected-action AST/behavior proof
Server-derived tenant/actor/fresh-auth	Partially inferred by token presence	Required action-structure and negative tests
SoD across all stages	Approval/preparer only	Full stage matrix and RBAC negatives
Atomic rollback	Historical mock examples only	Failure injection plus PostgreSQL transaction proof
Legacy evidence honesty	Not detected	Runtime-versus-legacy evidence classification
Gate refactor tolerance	Fragile string markers	Contract/AST/behavior-aware proof

Required test program

The twenty test categories in the attached prompt remain valid, with these additions:

- ordinary versus correction payroll-period monotonicity;
- read-only legacy permission cannot perform review;
- human and authorized-worker emission paths;
- transition table update/delete rejection;
- runtime row cannot omit actor/event/from-state evidence;
- legacy row cannot claim `VERIFIED` without full evidence;
- old combined action fails after cutover and after application rollback;
- same-key concurrent retry returns replay, while different-key competition returns conflict;
- generic event/outbox contains no person-level or payment-destination data;
- gate accepts secure delegated helpers and fails unsafe delegated helpers.

Files expected to change during implementation

Core:

- `prisma/schema.prisma`
- new additive payroll transition migration(s)
- `services/payroll/payroll-control.schemas.ts`
- `services/payroll/payroll-control.service.ts`
- preferably a focused `services/payroll/payroll-run-transition.service.ts`
- `services/events/business-event.schemas.ts` or a focused payroll/HRIS event-contract registry
- `services/hris/operational-time.service.ts`
- `services/controls/sensitive-action.service.ts`
- `lib/security/rbac-permissions.ts`
- `actions/payroll/payroll-control.actions.ts`
- `services/accounting/close-assurance-pack.service.ts`
- affected payroll/accountant/assurance read models
- payroll action panel, workbench, command center, and hook
- payroll readiness gate and mutation fixtures

Tests must be added in focused new files where practical to reduce conflict with the currently modified large suites.

Residual risks and non-claims

- Current source may change before Work Package 0 completes; re-run discovery from the controlled baseline.
- Existing finalized history cannot be upgraded to independently reviewed history without authentic evidence.
- A static green gate is not concurrency or rollback proof.
- Disposable PostgreSQL evidence is not production database evidence.
- No country pack, provider, filing authority, labor rule, tax rule, accessibility state, privacy program, or production release is certified by this report.
- Report trust currently has a live gate false negative that must be reconciled before cross-domain ratification.

- The current Jest execution path did not provide a bounded pass/fail result and must be repaired before implementation evidence is accepted.

Final approval decision

The Payroll Trust Spine plan is approved for controlled implementation after Work Package 0. The current payroll implementation is not approved, the real HIGH findings are not closed, and skill 012 cannot yet be honestly approved for downstream ratification.

POS/offline sync, purchasing/AP, AI execution, provider activation, billing, and growth implementation are not applicable to this repair except as preserved regression gates or later production dependencies. No code in those domains was modified by this run.